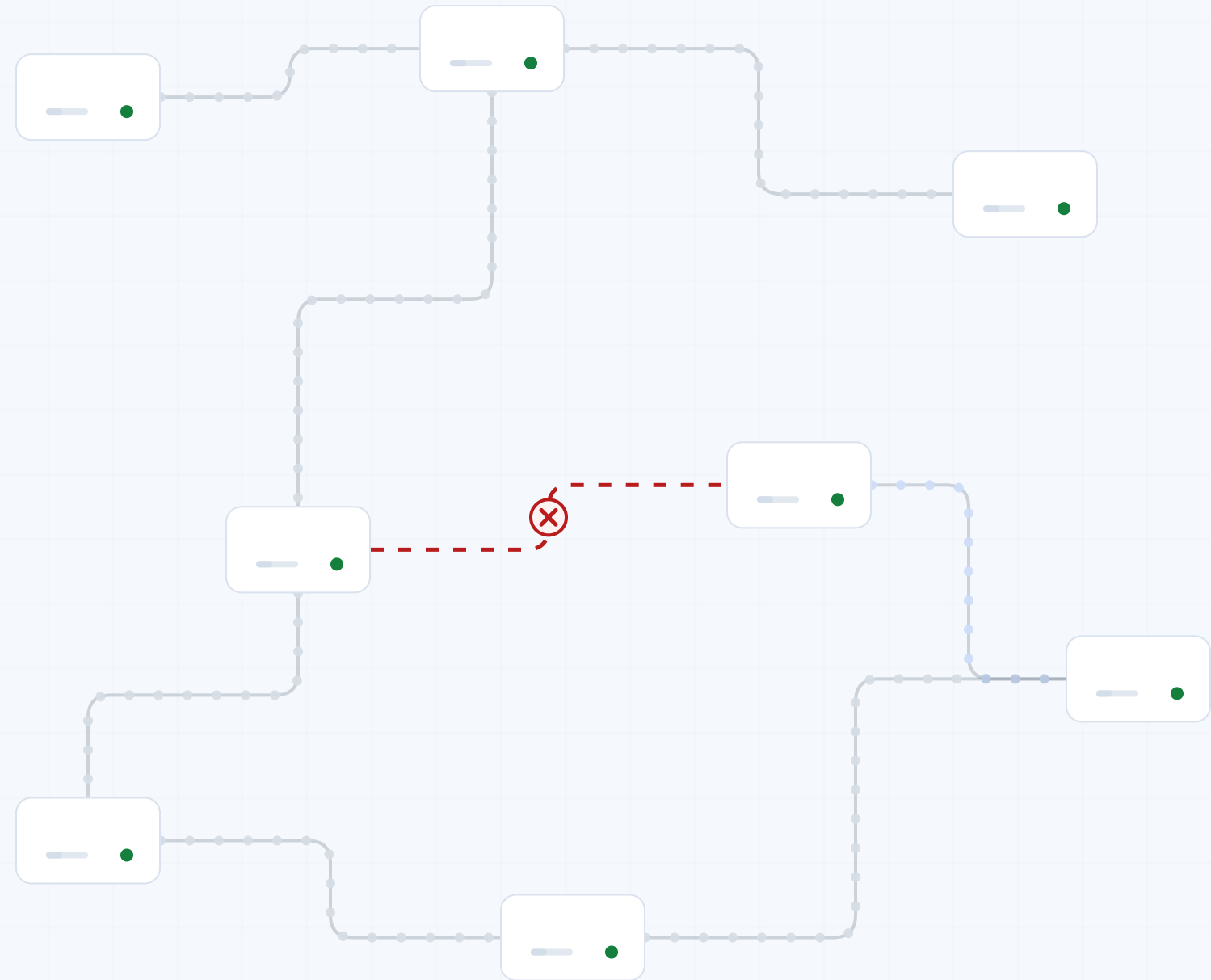


De Compose

a orquestación.

LA PREGUNTA

¿Y si una sola máquina no basta?



Siete pasos, una decisión.

SIETE PASOS

Contenedor, Compose, su límite, clúster, Pod con Deployment y Service, consenso, etcd; al final, **la decisión: réplicas o máquina más grande.**

LO QUE TE LLEVAS

- Compose por dentro y dónde se acaba
- Kubernetes básico: Pod, Deployment, Service
- Consenso desde cero: la mayoría del clúster completo



Lo que quedó de la S4.

DOS ANCLAJES

gRPC es RPC con contrato y deadline; el mismo mensaje pesa **38 bytes en JSON y 9 en Protobuf**.

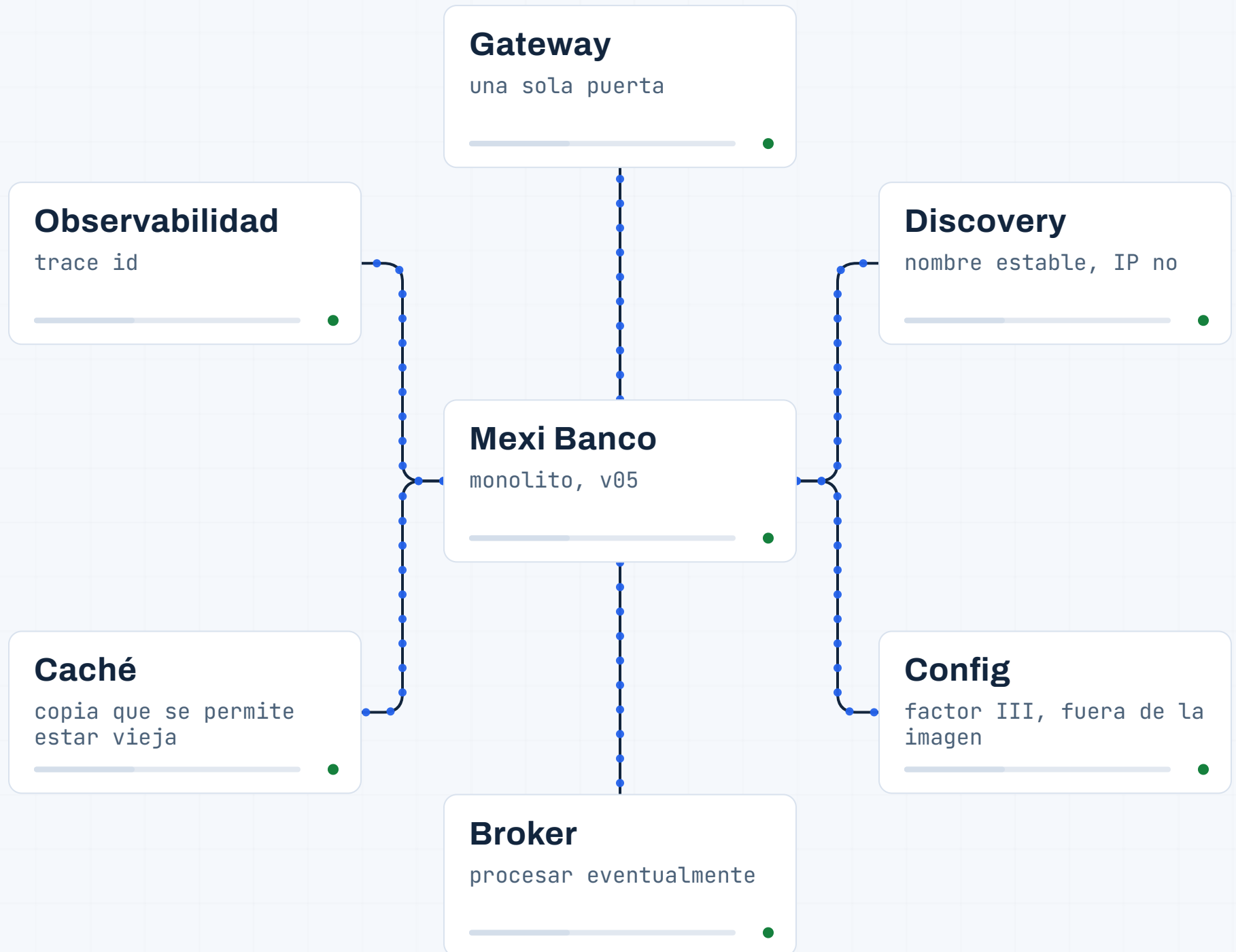
DE LA S4

gRPC por su nombre RPC, contrato obligatorio, deadline en cada llamada

HTTP/2 y HPACK cabeceras comprimidas; HTTP/1.1 las remanda completas

Seis piezas gateway, discovery, config, broker, caché, observabilidad

Cada servicio dueño de su almacén



FarmaYa,

en tres líneas.

FARMAYA

POST /pedidos/{id}/pagar llama a la pasarela (1 a 8 s). **El timeout de 3 s acaba de vencerse.** ¿Qué hace la app?

TRES LÍNEAS

elección A, B o C
porque el requisito que persigo
sacrifico lo que dejo atrás

POST **TIMEOUT DE 3 S VENCIDO**

A · Reintentar de inmediato

at-least-once; exige Idempotency-Key en la pasarela

Sacrifica

latencia extra y depender de que la pasarela deduplique

B · No reintentar

at-most-once; el cliente revisa su estado

Sacrifica

pedidos en limbo y soporte manual

C · Publicar en un broker

responde "en proceso"; consumidor idempotente

Sacrifica

la confirmación inmediata

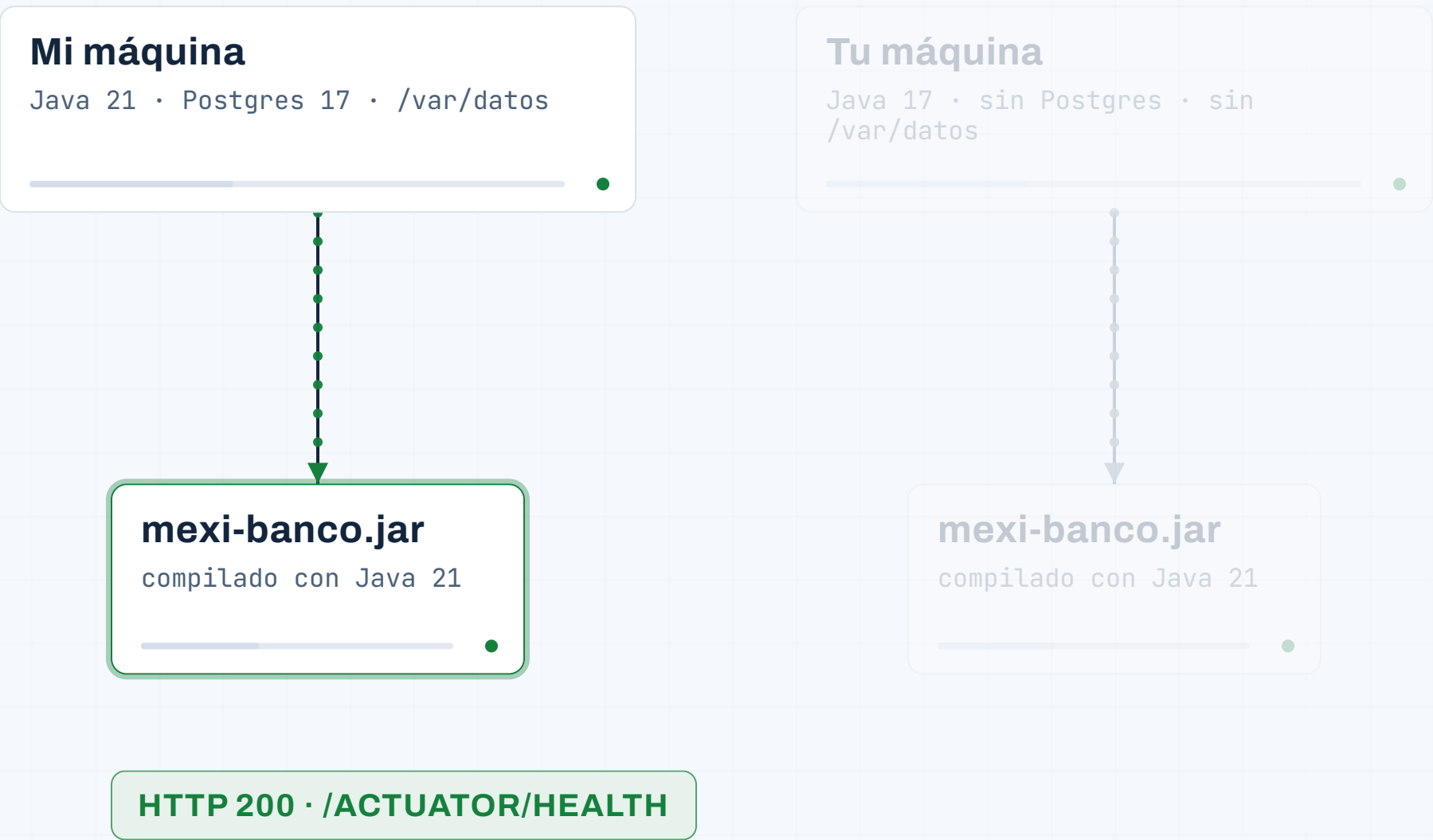
Corre aquí, falla allá.

DEPENDENCIAS DEL ENTORNO

La aplicación depende de **versiones, rutas y librerías de la máquina donde corre.**

LO QUE CAMBIA ENTRE MÁQUINAS

- Java 21 aquí, 17 allá
- Postgres instalada aquí, ausente allá
- La ruta del disco /var/datos no existe allá
- La frase "en mi máquina sí corre"



EL MISMO MEXI-BANCO.JAR EN LAS DOS

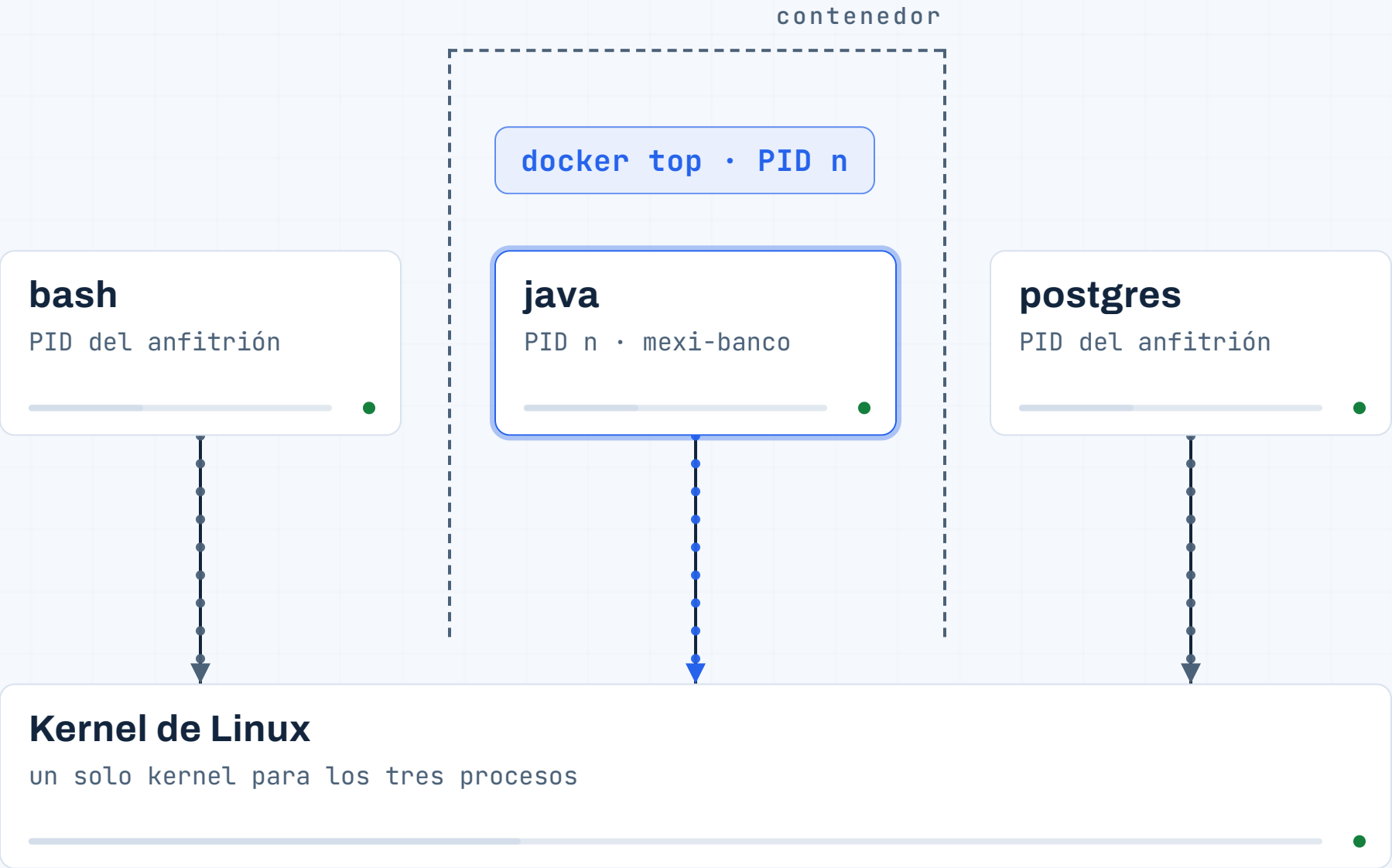
Un proceso con paredes.

CONTENEDOR

Un **proceso aislado** con todos los **archivos que necesita para correr**; comparte el kernel del anfitrión.

CUATRO PROPIEDADES

- Autocontenido** no depende de nada preinstalado
- Aislado** influencia mínima sobre el host y los demás
- Independiente** se borra uno sin tocar los otros
- Portátil** corre en cualquier lado



No es una máquina virtual.

MÁQUINA VIRTUAL

Un **sistema operativo entero con su propio kernel**; un contenedor es un proceso que comparte el kernel.

LA DIFERENCIA

- VM** kernel propio sobre un hipervisor
- Contenedor** kernel compartido, un proceso aislado
- Consecuencia** caben más aplicaciones en menos hardware

MÁQUINA VIRTUAL

Aplicación

mexi-banco.jar

SO invitado

kernel propio, drivers, programas

Hipervisor

emula el hardware

Hardware

CPU, memoria, disco

CONTENEDOR

Aplicación en contenedor

mexi-banco.jar y sus archivos

Kernel del anfitrión

compartido por todos los contenedores

Hardware

CPU, memoria, disco

Dos contenedores, un archivo.

DOCKER COMPOSE

Herramienta para definir y correr aplicaciones multicontenedor: un YAML y un comando.

MEXI BANCO EN COMPOSE

- services:** app y db, un contenedor por componente
- docker compose up** levanta los dos con un comando
- Sin Compose** dos docker run con red, puertos y orden a mano

DOCKER-COMPOSE.YML

```
services:
  app:
    build: .
    ports: "8080:8080"
  db:
    image: postgres:17-alpine
```

docker compose up --build

curl localhost:8080

app
creando

db:5432

db
creando

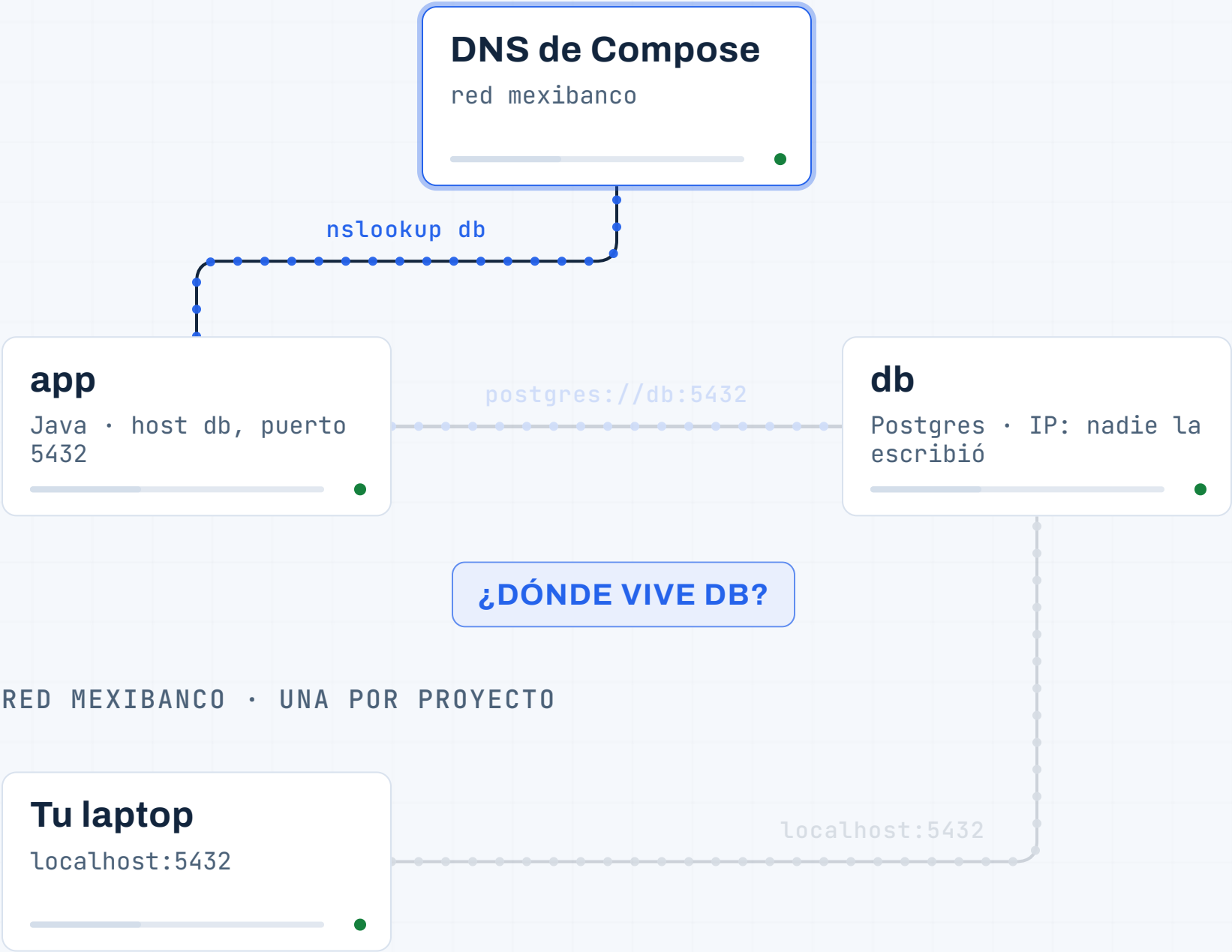
El nombre es la dirección.

RED POR PROYECTO CON DNS

Cada servicio es alcanzable por los demás de su red y **descubrible por el nombre de su servicio**.

EN MEXI BANCO

- Red mexibanco, una por proyecto
- Desde app postgres://db:5432, sin IP escrita
- Desde tu laptop localhost por el puerto publicado, 5432



Arrancó

no es listo.

DEPENDS_ON

Corto: solo garantiza que la dependencia **ya arrancó**, no que esté lista. Largo: admite condition.

LAS CONDICIONES

- Sin condition app conecta antes de tiempo y truena
- service_healthy espera a que db esté healthy
- Exige un healthcheck en db
- Las tres service_started, service_healthy, service_completed_successfully

app sin condition

depends_on: [db]

COMPOSE.YAML

depends_on:

db:

condition: service_healthy

app con condition

condition:
service_healthy

db

Up · starting · sin conexiones

Tres estados de salud.

HEALTHCHECK

Comando que decide si el contenedor está sano. Tres estados: **starting, healthy, unhealthy**.

EN MEXI BANCO

db pg_isready -U mexibanco cada 5 s
app wget /actuator/health cada 10 s
Compose el bloque healthcheck entra al YAML

db

recién creado · sin respuesta sana aún

ESTADO **STARTING**

pg_isready

-U mexibanco · cada 5 s

COMPOSE.YAML · DB

healthcheck:

test: pg_isready -U mexibanco

interval: 5s

retries: 5

`docker inspect --format '{{.State.Health.Status}}' mexi-banco-db-1`

Cuánto tarda en darse cuenta.

POR OMISIÓN · INTERVAL 30S · TIMEOUT 30S · RETRIES 3

VALORES POR OMISIÓN

interval 30s, timeout 30s, retries 3:
tres fallos consecutivos para
unhealthy, del orden de 90 s.

POR OMISIÓN Y EN MEXI BANCO

Dockerfile interval 30s · timeout 30s · retries 3
También start-period 0s · start-interval 5s
Mexi Banco db interval 5s · retries 5

FALLO 1 · 30 S

FALLO 2 · 60 S

FALLO 3 · 90 S

db dejó de responder · cada marca es un intento fallido

ESTADO **UNHEALTHY · 90 S**

El gafete

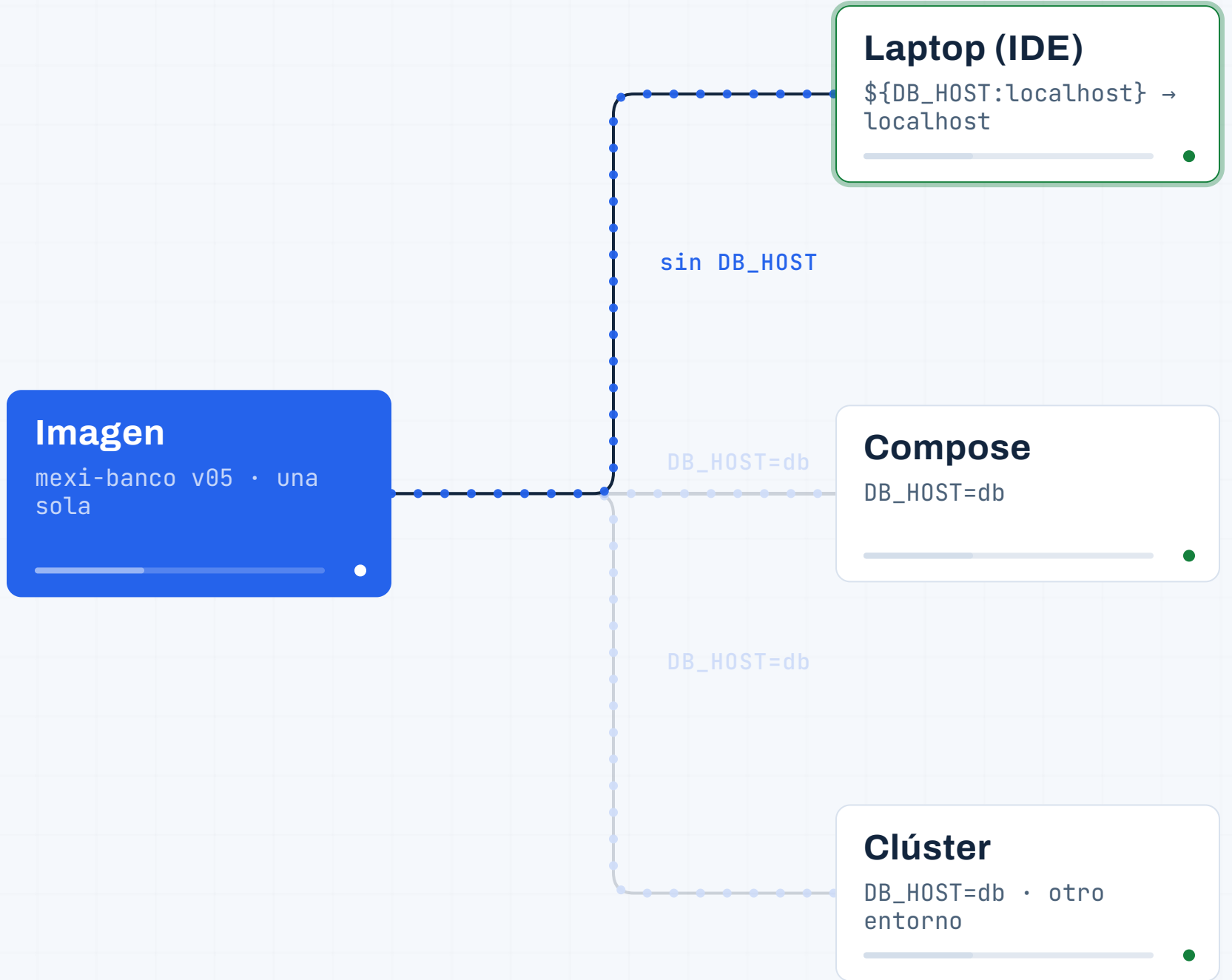
y el uniforme.

VARIABLES DE ENTORNO

La **misma imagen** corre en laptop, Compose y clúster; lo que cambia viaja por el entorno, no en la imagen.

EN EL COMPOSE Y EL IDE

compose environment: DB_HOST: db
application.yml \${DB_HOST:localhost} para el IDE
env_file saca las variables del YAML;
environment gana



Lo que sobrevive.

VOLUMEN

Existe **fuera del ciclo de vida** del contenedor: al destruirlo muere su capa de escritura, no el volumen.

EN MEXI BANCO

Montaje
mexibanco_datos:/var/lib/postgresql/data
down conserva el volumen
down -v borra el volumen y los datos



¿Qué garantiza service_healthy?

COMPROBACIÓN 1

app tiene depends_on: db con condition: service_healthy. ¿Qué garantiza **exactamente**?

OPCIONES

- A Postgres acepta conexiones
- B el contenedor existe, nada más
- C app y db comparten red
- D si db cae, app se reinicia

COMPOSE.YAML · APP

depends_on:

db:

condition: service_healthy

db

healthcheck: pg_isready

app

depends_on: db ·
service_healthy

ESTADO **HEALTHY**

Quién lo levanta.

POLÍTICA DE REINICIO

no, on-failure[:n], always o unless-stopped; **aplica solo si el contenedor vivió al menos 10 s.**

EN MEXI BANCO

restart: unless-stopped en app y en db

kill app vuelve en **5 s**

stop app no vuelve: lo paraste tú

Demonio de Docker

aplica restart cuando el proceso sale

vigila

app

Up · healthy

restart: unless-stopped

SI VIVIÓ AL MENOS 10 S

CUATRO POLÍTICAS

no nunca

on-failure[:n] solo con error

always siempre

unless-stopped salvo que lo pararas tú

Proceso, demonio y máquina.

TRES FALLAS DISTINTAS

Proceso: lo levanta la política;
demonio: **always y unless-stopped vuelven**; máquina apagada: nadie.

QUÉ SOBREVIVE

Proceso on-failure, always, unless-stopped
Demonio on-failure **no** vuelve
Máquina el demonio se apagó con ella



17 / LÍMITE

Un solo host.

LÍMITE DE COMPOSE

Compose soporta **despliegues de producción en hosts individuales**: su universo es una máquina.

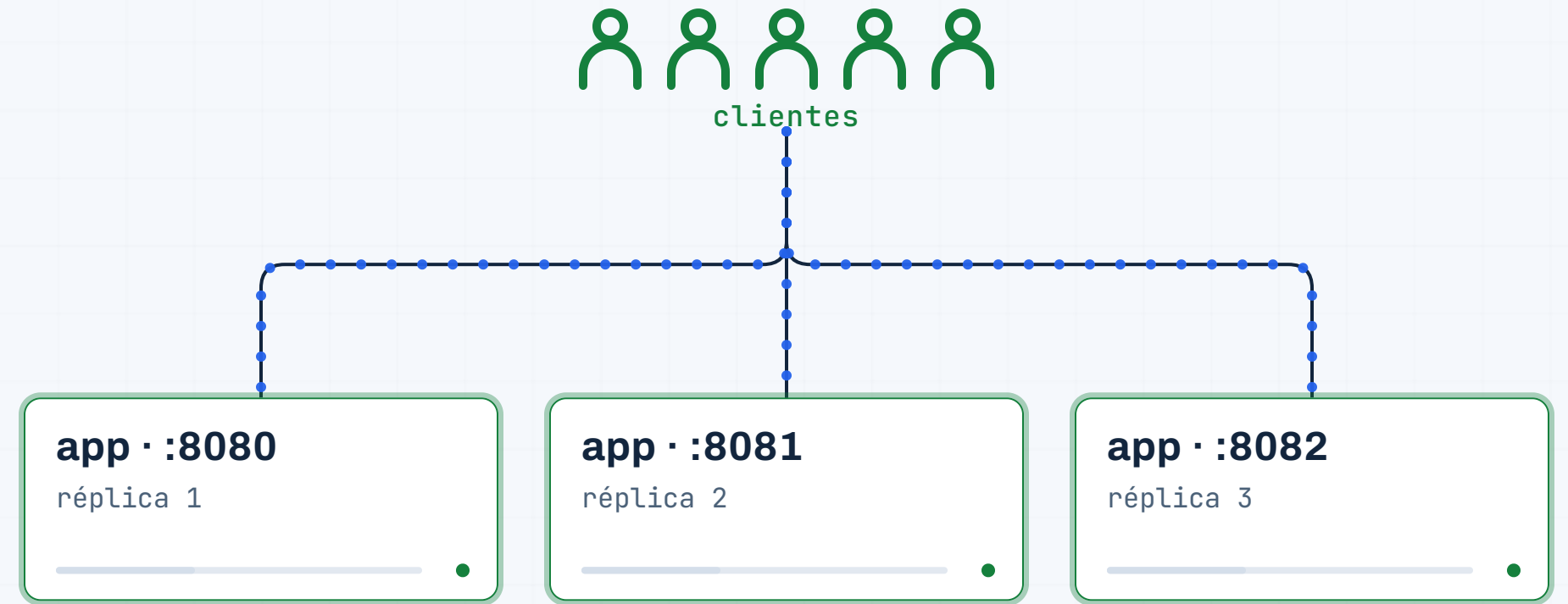
CON `--SCALE APP=3`

Tres puertos 8080, 8081, 8082, un host

Sin balanceo cada cliente elige puerto

Host caído caen las tres

Swarm "reprograma en otros nodos"



UN HOST · `DOCKER COMPOSE UP -D --SCALE APP=3`

SIN BALANCEADOR: NADIE REPARTE

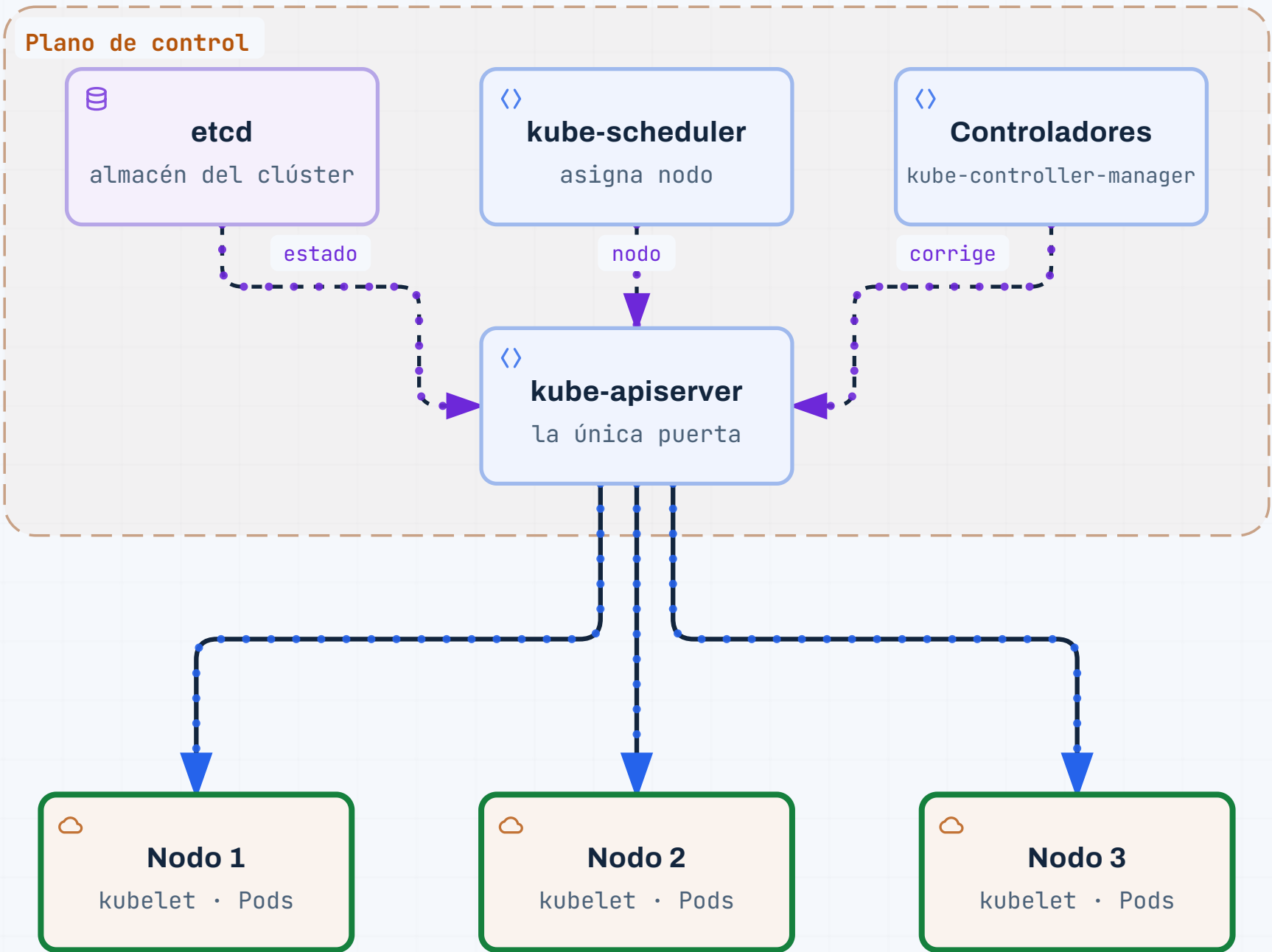
Varias máquinas.

CLÚSTER

Un **plano de control** más un conjunto de nodos que corren las aplicaciones en contenedores.

QUIÉN HACE QUÉ

Nodos hospedan los Pods; kubelet en cada uno
Plano de control gobierna nodos y Pods
Producción plano de control en **varias máquinas**
`kubectl get nodes` los enlista



En una frase.

KUBERNETES

Plataforma portátil, extensible y de código abierto para **gestionar cargas en contenedores**.

LA FRASE COMPLETA

Facilita configuración declarativa y automatización

K8s ocho letras entre la K y la s

Compose una máquina, hosts individuales

Kubernetes un clúster, varias máquinas

Docker Compose

una máquina

UN YAML, UN COMANDO

HOSTS INDIVIDUALES

UNA MÁQUINA

Kubernetes

un clúster

PORTÁTIL Y EXTENSIBLE

CÓDIGO ABIERTO

CONFIGURACIÓN DECLARATIVA

AUTOMATIZACIÓN

K8s = K + ubernete + s

Di qué quieres, no cómo.

ESTADO DESEADO

Describes lo que quieres y **Kubernetes cambia el estado real hasta alcanzarlo**, a un ritmo controlado.

CÓMO SE PIDE

Sí "quiero 3 copias de banco:1.4 y una dirección estable"

No "arranca esto en esta máquina"

kube-scheduler elige el nodo de cada Pod

Estado deseado

replicas: 3 · banco:1.4

compara

CUENTA

deseado

3

real

2 → 3

ajusta

HAY 2, QUIERO 3: CREA 1

banco-7f2a

nodo 1 · Running

banco-9c4d

nodo 2 · Running

banco-b81e

nodo 1 · nuevo

El termostato

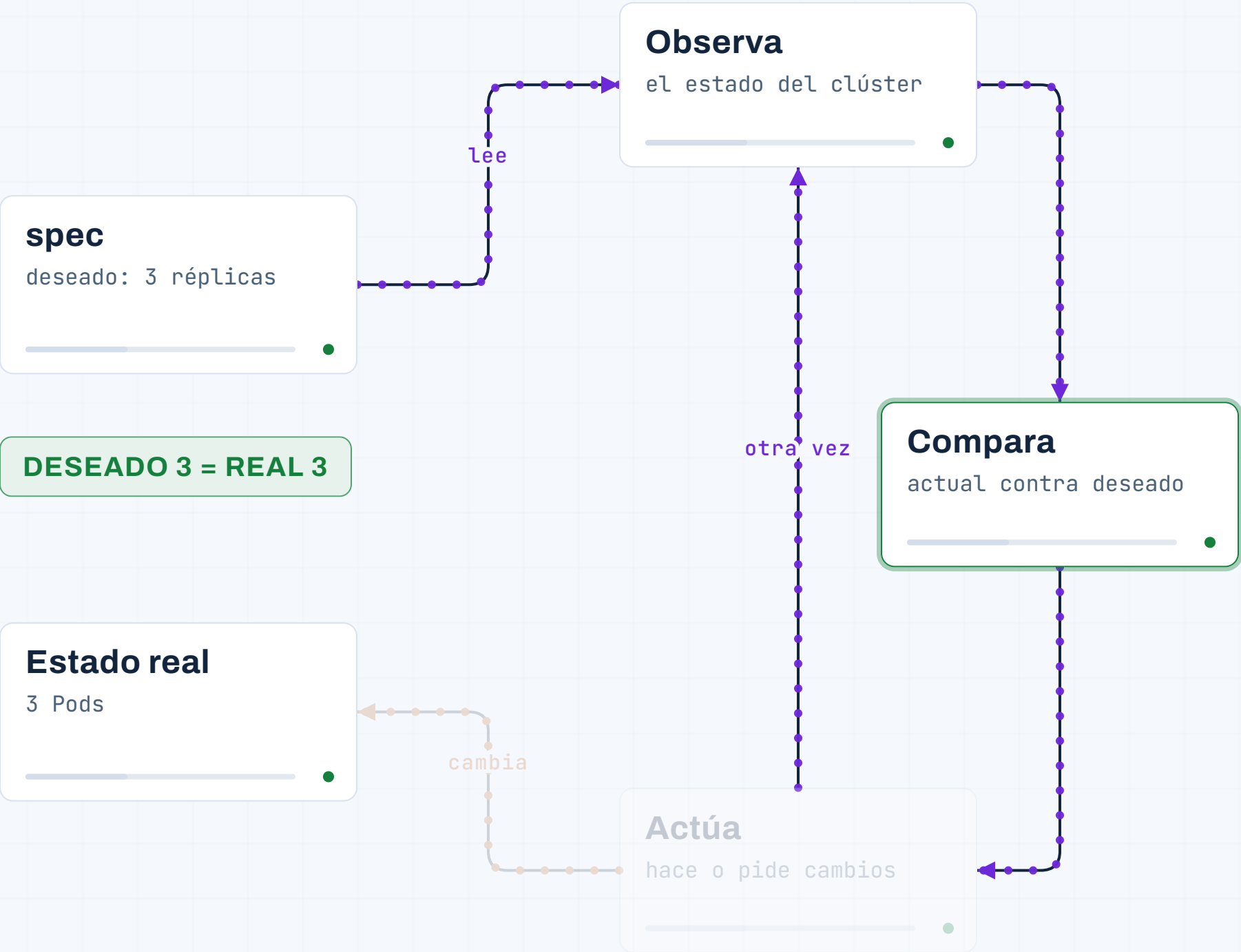
que nunca duerme.

CONTROLADOR

Bucle que observa el estado del clúster y **hace o pide cambios** para acercar el actual al deseado.

EL BUCLE

- Observa el estado del clúster
- Compara actual contra deseado
- Actúa hace o pide cambios
- Nunca termina los corre kube-controller-manager



La unidad mínima.

POD

La unidad de cómputo más pequeña:
uno o más contenedores que comparten red y almacenamiento.

LO QUE HAY QUE SABER

Una IP por Pod; sus contenedores la comparten
Efímero desechable; rara vez se crea a mano
Lo crea un Deployment, no una persona

Pod

mexi-banco-6b7a9-k2xq

Contenedor

java · banco:1.4

IP **10.244.1.7****Pod**

mexi-banco-6b7a9-m8rt · dos contenedores

Contenedor

java · banco:1.4

Contenedor

agente de logs

IP **10.244.1.9 · UNA SOLA**

Quien cuenta las copias.

DEPLOYMENT

Gestiona Pods sin estado; **debajo, el ReplicaSet mantiene estable el número de Pods** creando y borrando.

EN K8S/APP.YAML

replicas: 3 el número deseado va en `.spec.replicas`

template la plantilla con la que se crea cada Pod

ReplicaSet crea y borra Pods hasta llegar al número

Deployment

mexi-banco · replicas: 3

ReplicaSet

mexi-banco-6b7a9

REPLICASET DESEADO 3 · REAL 3

Pod

mexi-banco-6b7a9-k2xq

Pod

mexi-banco-6b7a9-m8rt

Pod

mexi-banco-6b7a9-p7vd

Nunca

vuelve el mismo.

REEMPLAZO DE UN POD

Se programa una sola vez; **nunca se reprograma a otro nodo: se reemplaza por un Pod nuevo casi idéntico** con otro nombre y otra IP.

CUANDO MUERE EL NODO

- 1 el kubelet deja de renovar su Lease
- 2 el nodo queda NotReady
- 3 el ReplicaSet ve 2, quiere 3
- 4 nodo, nombre e IP nuevos

Nodo A

kubelet · Lease renovado

Pod

mexi-banco-6b7a9-k2xq ·
10.244.1.7

Nodo B

kubelet · Lease renovado

Pod

mexi-banco-6b7a9-m8rt

Pod

mexi-banco-6b7a9-p7vd

CONTRASTE **CRASHLOOPBACKOFF: MISMO POD, RESTARTS SUBE**

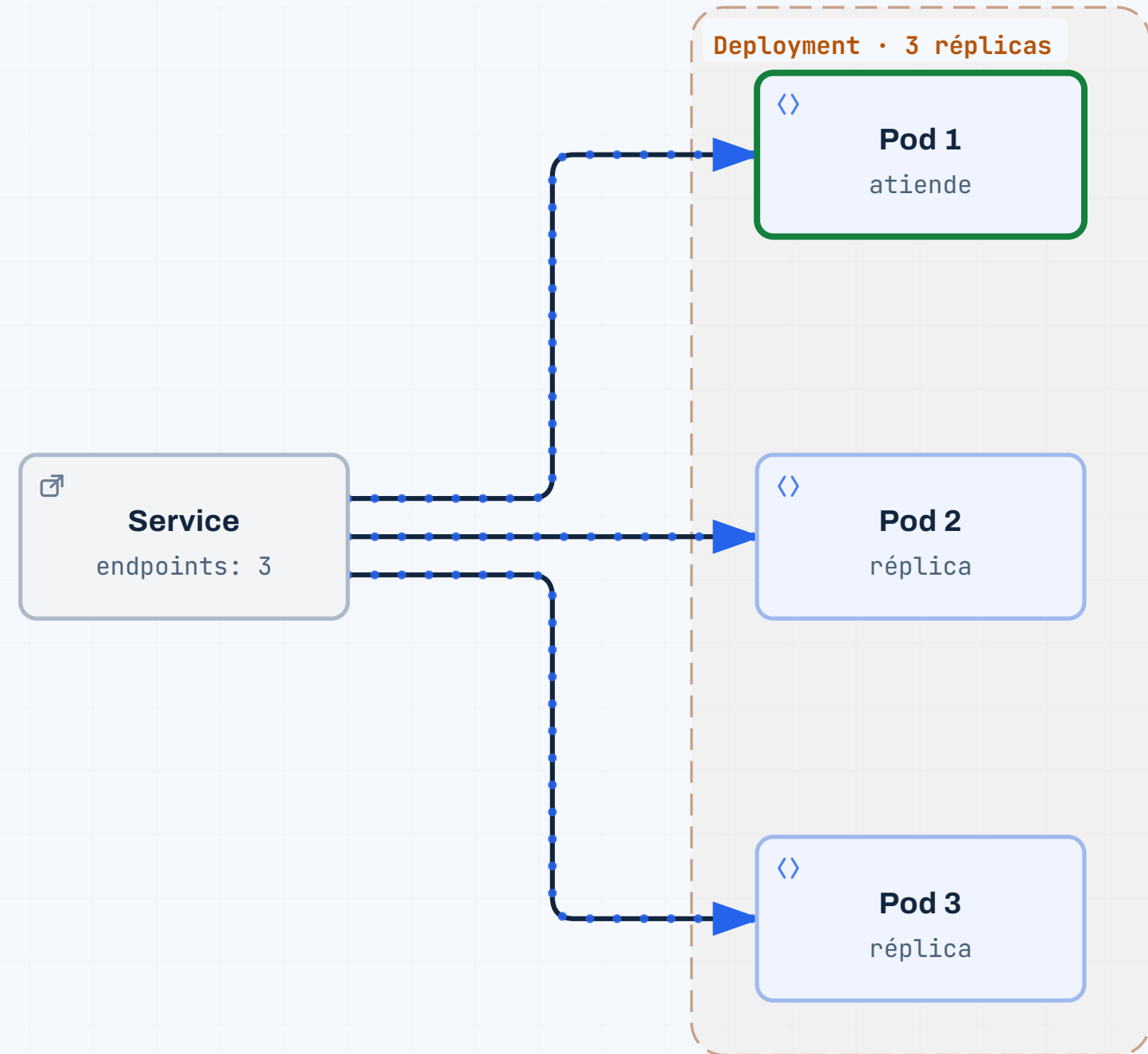
Una puerta estable.

SERVICE

Expone una aplicación que corre en uno o más Pods: **los encuentra por etiquetas y mantiene la lista al día.**

LO QUE DA

Nombre DNS estable: mexi-banco, no una IP
Endpoints por selector de etiquetas, no por IP
Nadie se conecta a un Pod; se conectan al Service



¿Qué predices?

COMPROBACIÓN 2

Un Deployment de Mexi Banco con 3 réplicas; corres `kubectl delete pod` sobre uno. ¿Qué predices?

OPCIONES

- A el mismo Pod, con el mismo nombre
- B un Pod nuevo, otro nombre, AGE 0
- C restaura el Pod de un respaldo
- D reinicia el clúster

Service

mexi-banco · 10.96.0.12

Pod

mexi-banco-6b7a9-k2xq

¿?

mexi-banco-6b7a9-m8rt

Pod

mexi-banco-6b7a9-p7vd

`kubectl delete pod mexi-banco-6b7a9-m8rt`

DEPLOYMENT · REPLICAS: 3

Tres copias, dos verdades.

PLANO DE CONTROL · 3 COPIAS DEL ESTADO

FALLO PARCIAL

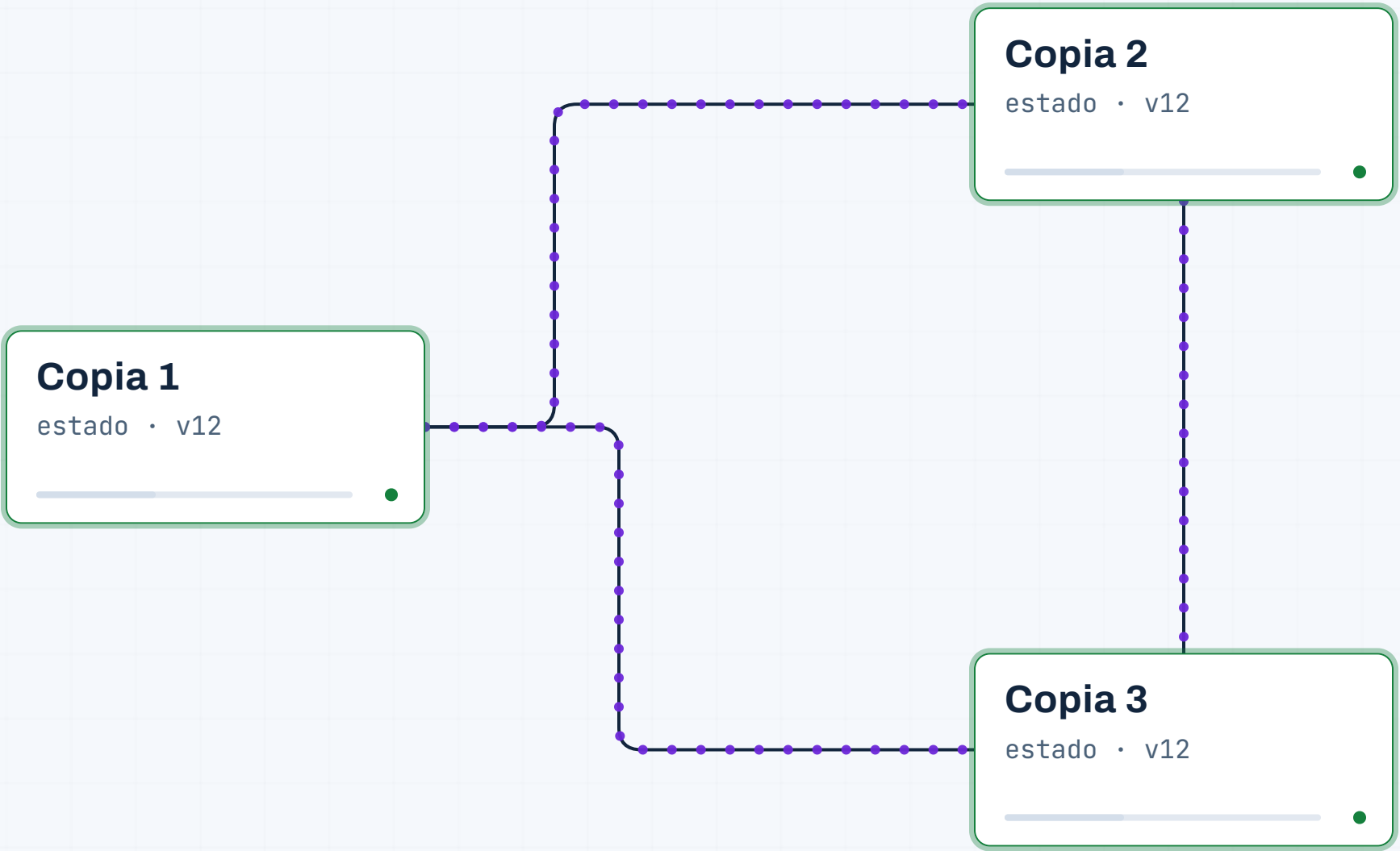
Tres copias de la verdad y la red se corta: **cada lado puede aceptar órdenes distintas**. Dos verdades.

LO QUE HAY QUE IMPEDIR

Lado A acepta "crea un Pod"

Lado B acepta "borra el Deployment"

Producción el plano de control corre con **3 o 5 copias**



¿Cuándo está

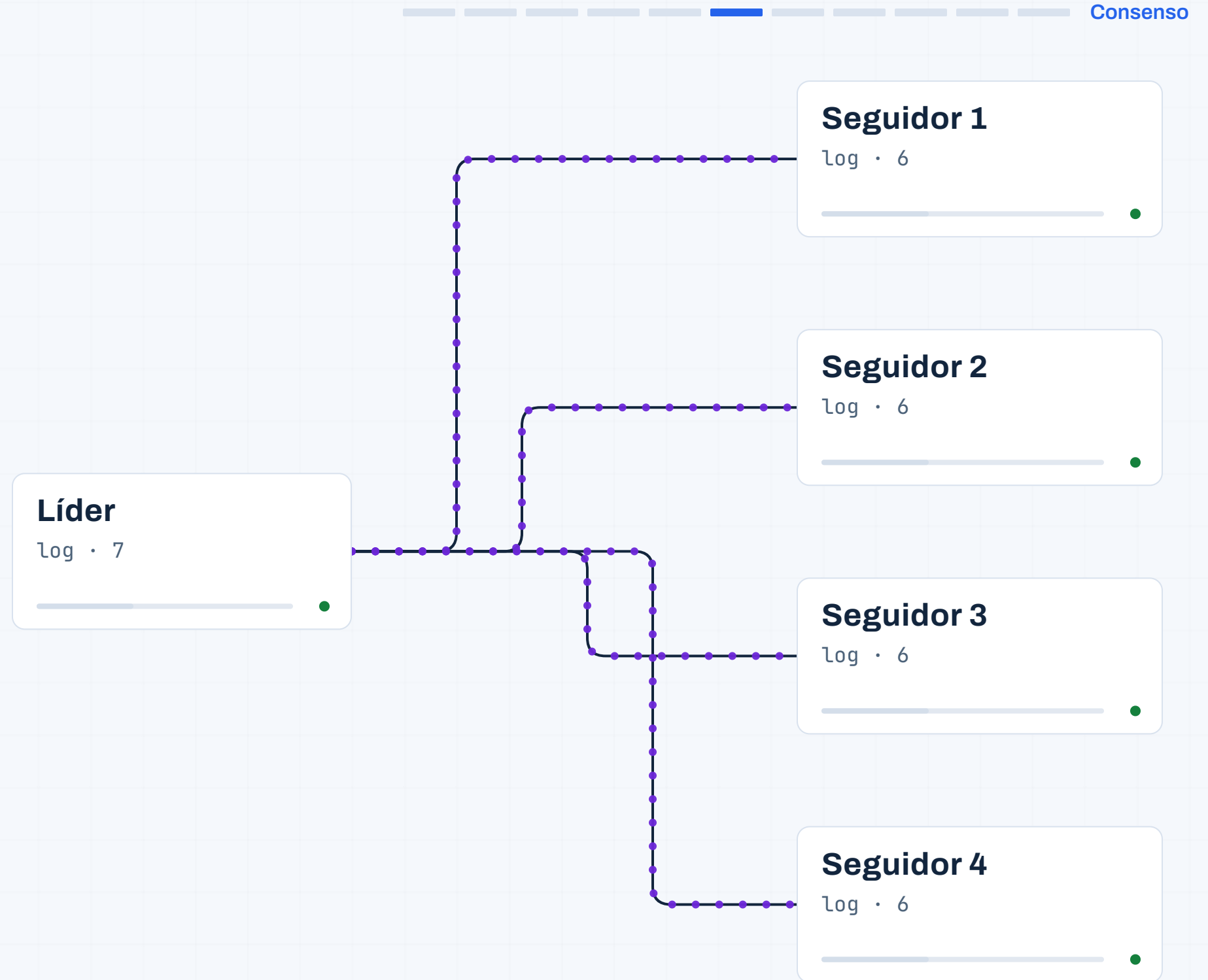
comprometida?

PREGUNTA DE LA SEMANA

Raft, 5 servidores. El líder manda una entrada al log. ¿Cuándo la considera **comprometida**?

ELIGE UNA

- A cuando la escribe en su propio log
- B cuando la mayoría, 3 de 5, ya la replicó
- C cuando los 5 la replicaron
- D cuando el cliente recibe la respuesta



La mayoría

del clúster completo.

CONSENSO

Máquinas que trabajan como un grupo coherente y **sobreviven al fallo de algunos miembros.**

MAYORÍA, CON LA DEFINICIÓN EXACTA

5 servidores toleran el fallo de 2 cualesquiera

Se cuenta sobre los 5 del clúster completo, **no sobre los vivos**

2 vivos de 5 no deciden nada

Servidor 1
seguidor

Servidor 2
seguidor

Servidor 3
seguidor

CLÚSTER DE 5

mayoría

tolera

3 de 5
2 caídos

Servidor 4
seguidor

Servidor 5
seguidor

MAYORÍA A · 1, 2, 3 · 3 DE 5

Un latido, y si falta, elección.

HEARTBEAT

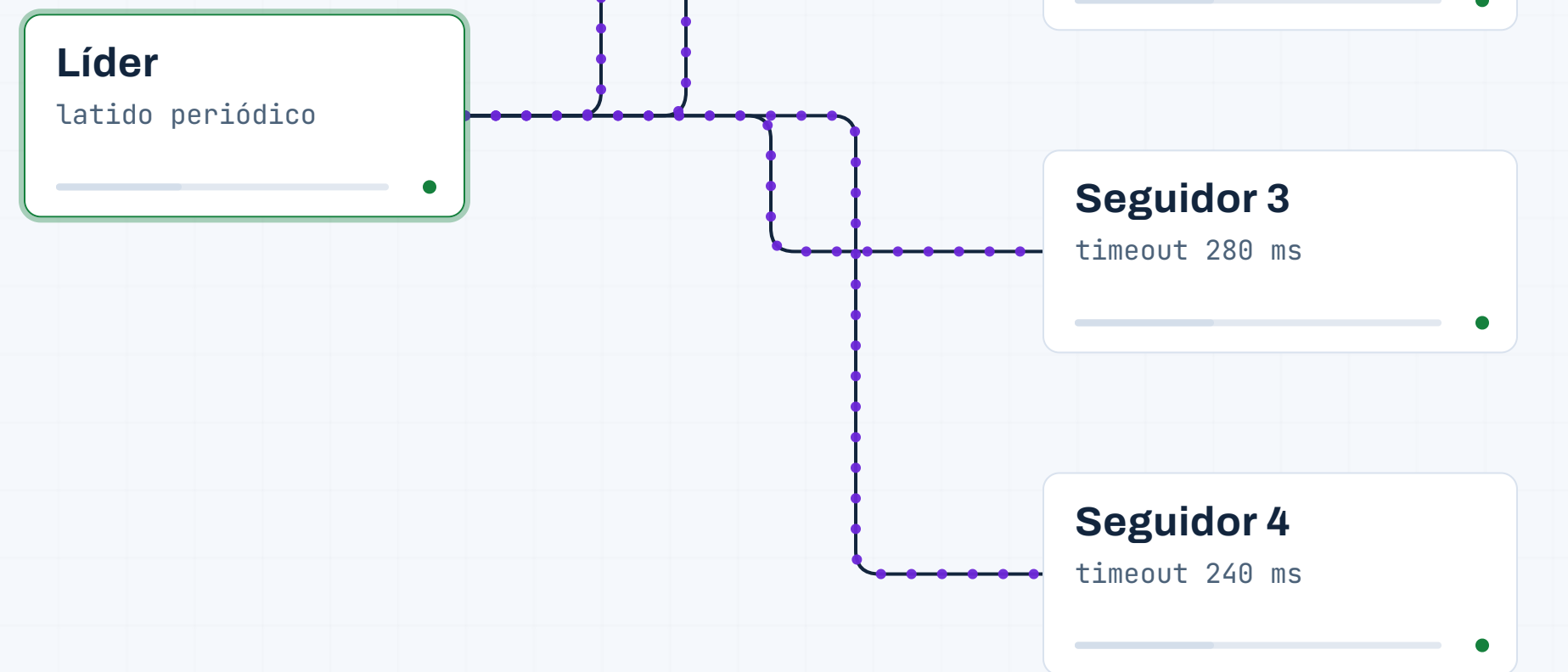
El líder manda latidos periódicos; quien no los oye en su timeout de elección **se postula y pide votos**.

CÓMO SE ELIGE

Timeout aleatorio 150 a 300 ms, distinto en cada servidor

Un voto por servidor y por término

Votos divididos nadie gana; otra ronda con timeout nuevo



Cuánto dura sin líder.

REGLA DE TIEMPOS

Difusión << timeout de elección <<
tiempo entre fallos: **0.5 a 20 ms, 10 a 500 ms, meses.**

MEDIDO EN EL PAPER

Peor caso **513 ms** con 50 ms de aleatoriedad
Sin aleatoriedad más de 10 s por votos
divididos
Timeout muy bajo cambios de líder
innecesarios

DIFUSIÓN · 0.5 A 20 MS

ELECCIÓN · 10 A 500 MS

ENTRE FALLOS · MESES

ESCALA LOGARÍTMICA · ×10 POR TRAMO

TIEMPO SIN LÍDER

peor caso **513 ms**
aleatoriedad **50 ms**

TIMEOUTS DISTINTOS · UNO GANA

Dónde vive el acuerdo.

ETCD

Almacén llave-valor consistente y de alta disponibilidad, **respaldo de todos los datos del clúster.**

DENTRO DE KUBERNETES

kube-apiserver la única puerta hacia etcd

kubectl get pods lee de aquí

Raft por líder un seguidor **reenvía al líder**

No está en etcd no existe en el clúster

kubectl

apply · get pods

kube-apiserver

la única puerta

etcd 1

seguidor

etcd 2

líder · atiende consenso

etcd 3

seguidor

3 MIEMBROS · RAFT

Impares, y no más de siete.

QUÓRUM

$(n/2)+1$. Un clúster impar **tolera los mismos fallos** que uno par, con menos nodos; no más de siete.

FALLOS QUE TOLERA

- 3 miembros mayoría 2, tolera 1
- 4 miembros mayoría 3, tolera 1
- 5 miembros mayoría 3, tolera 2
- Kubernetes impar; cinco en producción

n = 3
mayoría 2 · tolera 1



QUÓRUM $(N/2)+1$
mayoría 2 de 3
tolera 1 fallo

n = 4
mayoría 3 · tolera 1



n = 5
mayoría 3 · tolera 2



TECHO: 7 MIEMBROS

Réplicas o máquina más grande.

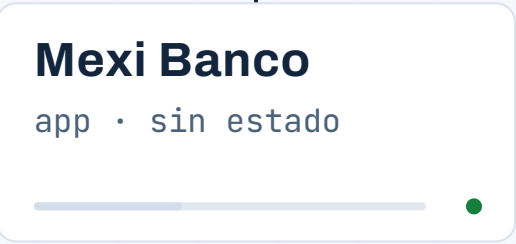
ESCALAR

Cambiar el **número de réplicas** (horizontal) o los **recursos de cada una** (vertical).

QUÉ EXIGE CADA UNA

Horizontal procesos **sin estado** y algo que reparta

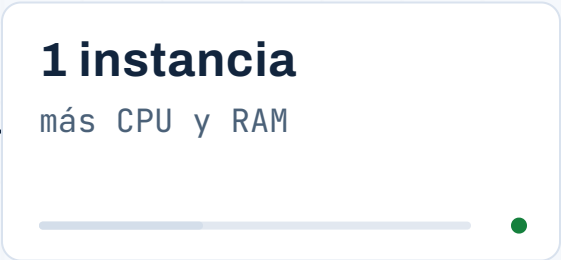
Vertical un solo punto de falla; no cambia código **etcd** cinco con **discos rápidos**, no diez lentos



A · HORIZONTAL



B · VERTICAL



La decisión

de hoy.

LA DECISIÓN

Mexi Banco recibe **el triple de tráfico** en la autorización de tarjeta:
¿réplicas o máquina más grande?

EN TRES LÍNEAS

Elección una de las dos

Porque la razón, con el requisito enfrente

Sacrificio lo que pierdo al elegir

Mexi Banco

autorización · tráfico x 3

ELECCIÓN

PORQUE

SACRIFICIO

Réplicas

horizontal · n copias
iguales

Máquina más grande

vertical · 1 instancia

Descanso.

Mexi Banco arriba.

DESCANSO

Al regresar, Mexi Banco corriendo en su máquina: **clone** y **compose up**.

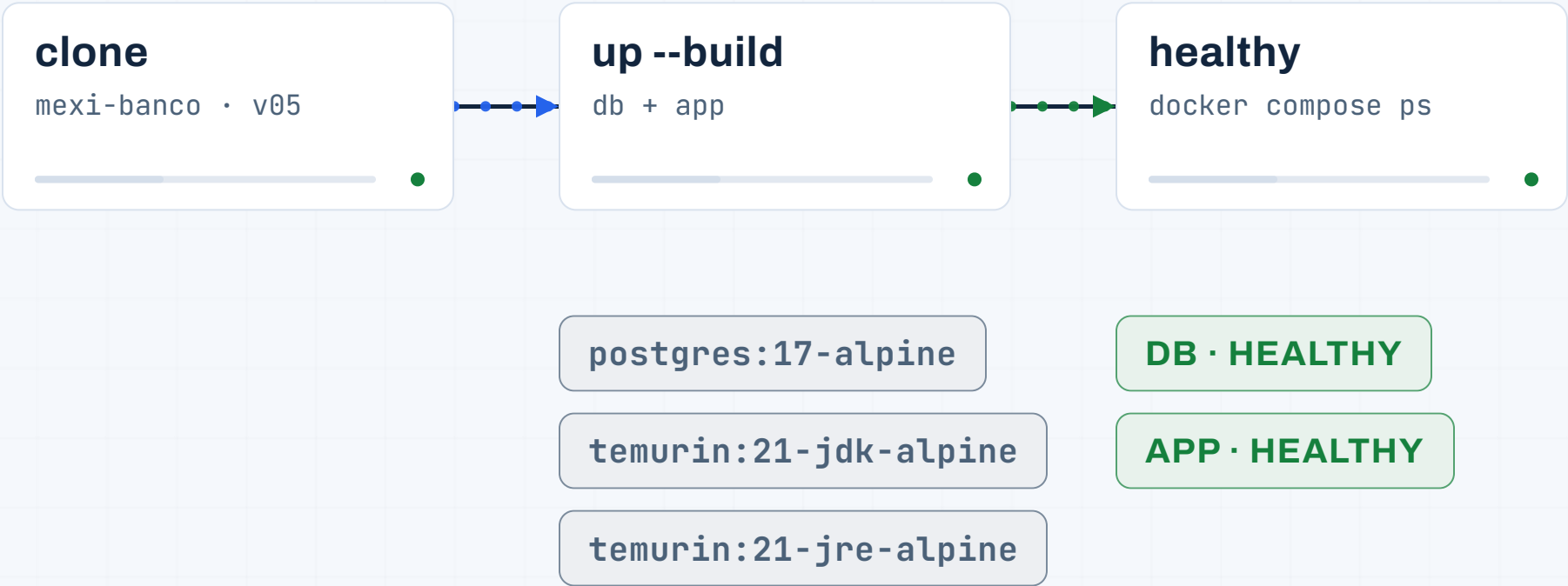
MIENTRAS TANTO

git clone --branch v05
github.com/OscarRuiz21/mexi-banco

docker compose up --build la primera vez
descarga tres imágenes

Si algo truena lo vemos al regresar, **no lo peleen solos**

MEXI-BANCO · V05 · DB + APP



Un banco, cinco piezas.

UN MONOLITO

Cinco piezas en **un solo deployable**:
un proceso Java y una base de
datos.

CUATRO DE LAS CINCO

- Cuenta** CLABE y saldo
- Movimiento** asiento inmutable: el ledger solo crece
- Transferencia** interna, una sola transacción
- SPEI** **externo, lento e irreversible**

MEXI-BANCO V05 · UN SOLO DEPLOYABLE · UNA SOLA BASE

Cuenta

CLABE, saldo

Movimiento

asiento inmutable

Transferencia

interna · 1 transacción

SPEI

externo, lento,
irreversible

Notificación

stub · escribe en el
log

Qué se puede pedir.

CUATRO ENDPOINTS

Abrir, consultar, transferir, SPEI. Y /actuador/health: lo que healthcheck y probes **preguntan**.

REST, COMO EN LA T01

- POST /cuentas
- GET /cuentas/{clabe}
- POST /transferencias
- POST /spei + Idempotency-Key



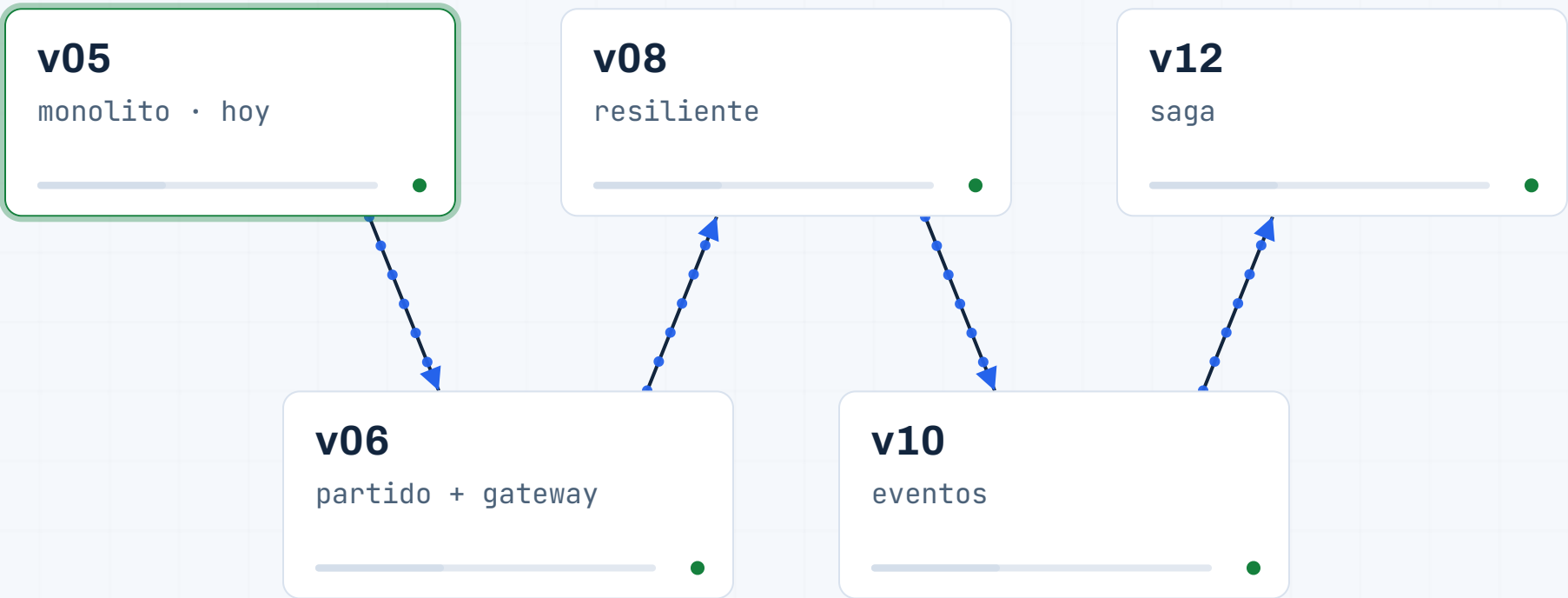
Un sistema que crece.

UNA PIEZA POR SESIÓN

Hoy monolito. Cada sesión agrega **la pieza que ese día se explica:** partido, resiliente, eventos, saga.

LO QUE VIENE

- v06 se parte en tres; gateway y discovery
- v08 resiliencia y trazas
- v10 eventos y notificaciones reales
- v12 la saga del SPEI



Tres hitos,

con tus manos.

CON TUS MANOS

Arriba y respondiendo, las propiedades de Compose, la idempotencia otra vez. **Cada hito termina con algo que se ve.**

ENTREGA

docker compose ps ambos servicios healthy
Segundo SPEI misma clave, sin doble cobro
README las tres preguntas de cierre

Hito 1 · Arriba y respondiendo

`compose up --build` · abrir y consultar una cuenta

Hito 2 · Propiedades de Compose

`network inspect` · transferencia · `stop`
`app: no vuelve` · `start app`

Hito 3 · Idempotencia otra vez

misma SPEI, misma clave, un solo cobro

Se repara solo.

AUTOCURACIÓN

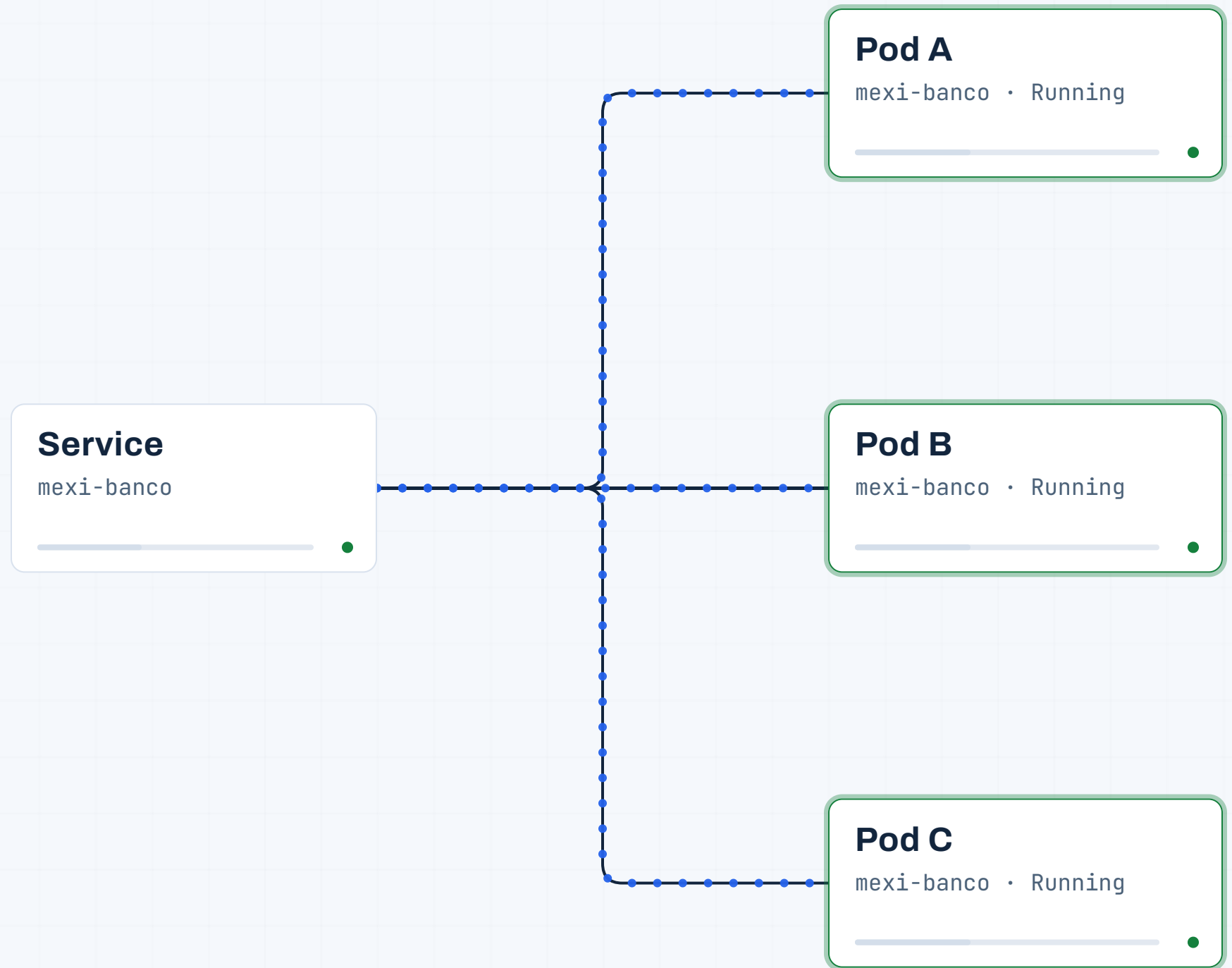
El Deployment no revive el Pod: crea uno **nuevo** porque nota que falta una copia.

LO QUE VAN A VER

Mismo Deployment 3 réplicas declaradas en k8s/app.yaml

Pod nuevo otro nombre, AGE en segundos

exec kill 1 RESTARTS sube, el nombre no cambia



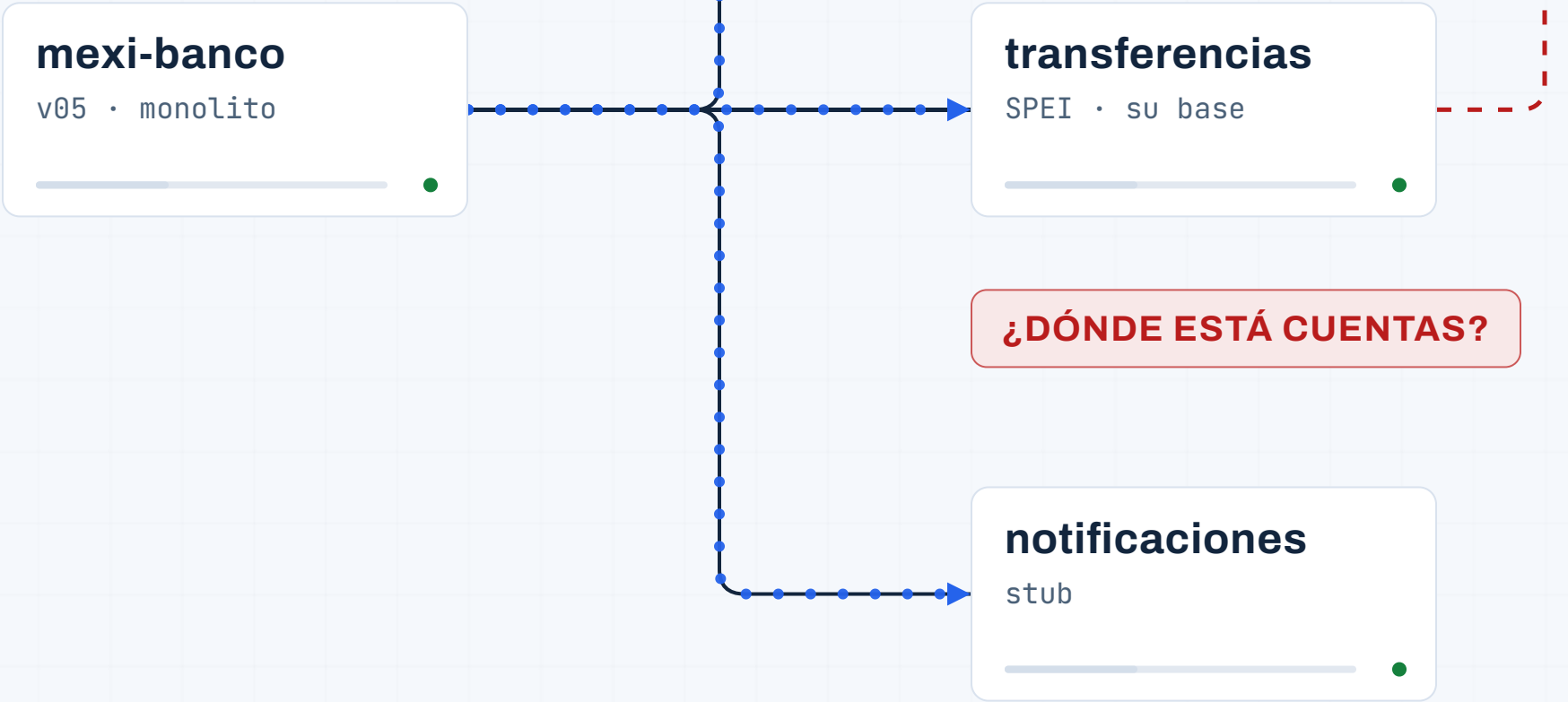
Para la S6.

PARA LA S6

Raft a fondo, consistencia y CAP.
Mexi Banco se parte en tres y la transferencia **deja de funcionar**.

LO QUE TRAES

Lectura 5 Brewer, CAP Twelve Years Later (2012)
Mexi Banco corriendo en Compose, como hoy



Ocho preguntas

que ya puedes responder.

OCHO PREGUNTAS

Si puedes contestarlas sin notas, **la sesión quedó**. Si una no, esa es la que repasas con el estudio.

COMPOSE

- 1 ¿Cómo encuentra la aplicación a la base de datos si nadie le dijo su IP?
- 2 ¿Qué pasa si la aplicación arranca antes que la base de datos?
- 3 ¿Quién vuelve a levantar un contenedor que se cayó solo?
- 4 ¿Hasta dónde llega Compose y por qué una sola máquina es el techo?

KUBERNETES Y CONSENSO

- 5 ¿Qué significa declarar el estado deseado en vez de dar órdenes?
- 6 ¿Qué pasa exactamente, paso a paso, cuando muere un Pod?
- 7 ¿Cómo elige Raft un líder con latidos y tiempos de espera aleatorios?
- 8 ¿Cuántas réplicas de etcd conviene tener y por qué número impar?

Fuentes

de la sesión.

FUENTES

Todo lo de hoy sale de fuentes **primarias**: la documentación oficial de Docker, Kubernetes y etcd, el paper de Raft y 12factor.net.

DOCUMENTACIÓN

docs.docker.com contenedor, Compose, networking, healthcheck, volúmenes, restart

kubernetes.io/docs architecture, controller, Pods, Deployment, ReplicaSet, Service, etcd

etcd.io/docs/v3.6 faq, tuning, hardware

12factor.net factor III, configuración; factor VI, procesos

PAPERS

Ongaro y Ousterhout, 2014 In Search of an Understandable Consensus Algorithm (Raft); raft.github.io, con el simulador

Brewer, 2012 CAP Twelve Years Later: How the Rules Have Changed; Lectura 5

Ya lo comprobaste.

EL PRINCIPIO

Compose asume que la máquina no falla; **Kubernetes** asume que sí, y automatiza la reacción.

ANTES DE CERRAR

Ticket de salida ¿acertaste tu predicción? ¿qué fue lo más confuso?

Lectura Raft §1 a 5: 2 preguntas + 1 hallazgo

Próxima S06 · Raft a fondo, consistencia y CAP;
Mexi Banco en tres

